Technical Design Document: AERIS

An Explainable Reliability Intelligence Engine for Event-Driven Systems

Author: Utkarsh Dubey

Role: Lead Architect / Researcher

Version: 1.0.0

Date: May 2025

1. Introduction

Modern cloud-native ecosystems rely heavily on **Event-Driven Architectures (EDA)** to maintain decoupling and scalability. However, as systems grow in complexity, "knowing a service is down" is no longer enough. We need to know *why* it is failing, *how* it is failing, and if the current failure pattern represents a structural degradation of the system.

AERIS (Audit & Explainable Reliability Intelligence System) is designed to shift the paradigm from reactive monitoring to **structured reliability reasoning**. It treats failures not as transient logs, but as immutable analytical data points to derive deterministic severity and explainable operational drift.

2. Problem Statement

Existing observability tools (Prometheus, Grafana) excel at telemetry but fail at **behavioral interpretation**.

- **Transient Context:** Failures are often treated as ephemeral alerts. Once a retry succeeds, the "near-miss" is forgotten, hiding systemic instability.

- **Threshold Myopia:** Alerting is usually binary (Up/Down) or threshold-based (CPU > 80%). It lacks the nuance to detect **Retry Pressure** or **Failure Clustering**.
- **The Black-Box Gap:** AI-driven observability often provides "Anomaly Scores" without explaining the underlying logic, leading to "alert fatigue" and lack of trust from SREs.
- **Undetected Drift:** Systems often experience "Boiling Frog" syndrome—a gradual increase in dead-letter rates that goes unnoticed because it stays just below traditional thresholds.

3. Objectives

- **Immutable Persistence:** Store every failure event in a tamper-proof state for historical auditing.
- **Deterministic Severity:** Replace "Low/Medium/High" guesswork with a rule-based engine that evaluates retry exhaustion and clustering.
- **Explainable Drift Detection:** Quantify how "Current" system behavior deviates from a "Baseline" using statistical windowing.
- **Confidence-Aware Scoring:** Ensure reliability reports include a confidence metric based on sample size to prevent false positives in low-traffic periods.

4. System Overview

AERIS functions as an intelligence layer sitting atop existing event-driven backends. It intercepts failure signals, processes them through three specialized intelligence layers, and provides an explainable output via a CLI and Dashboard.

Functional Layers:

1. **Event Intelligence:** Ingests failures, tracks retries, and identifies terminal "Dead Events."
2. **Severity Intelligence:** Applies a logic-gate model to classify the risk of an event.

3. **Drift Intelligence:** Compares time-series windows to detect operational degradation.

5. Architecture Diagram

The following diagram illustrates the flow from the external microservices into the AERIS intelligence core.

```
graph TD
    subgraph "External Ecosystem"
        S1[Microservice A] -->|Failure Event| IG[Event Ingestor]
        S2[Microservice B] -->|Failure Event| IG
    end

    subgraph "AERIS Core"
        IG --> ES[(Immutable Event Store - SQLite)]
        ES --> SE[Severity Engine]
        ES --> DE[Drift Analyzer]

        subgraph "Logic Engines"
            SE -->|Rule Matching| RB[Deterministic Rules]
            DE -->|Window Stats| WA[Statistical Comparator]
        end
    end

    subgraph "Output Layer"
        SE --> OUT[Intelligence API]
        DE --> OUT
        OUT --> CLI[AERIS CLI]
        OUT --> DB[Web Dashboard]
    end
```

6. Core Design & Approach

Why Determinism over Probabilistic ML?

In reliability engineering, **reproducibility is king**. If an SRE asks "Why was this marked Critical?", a black-box ML model might say "The weights shifted." AERIS, however, provides a **traceable logic path**: *"This was CRITICAL because it reached Max Retries (5/5) and occurred within a 10ms cluster of 50 similar events."*

Key Decision Matrix

Feature	Traditional Monitoring	AERIS Approach
Data Integrity	Ephemeral / Log-rotated	Immutable Event Store
Logic	Threshold-based (Binary)	Rule-based (Deterministic)
Context	Single-point snapshots	Windowed behavioral drift
Explainability	None (Alert only)	Explicit Logic Chains

7. Algorithm & Logic Design

A. Severity Derivation Algorithm

The severity S is computed based on three primary vectors: Retry Count (R), Terminal State (T), and Cluster Density (D).

1. **Step 1:** Receive Event (e).
2. **Step 2:** Check if e is a Dead Letter ($T = true$). If yes, base severity starts at **HIGH**.
3. **Step 3:** Evaluate R. If $R \geq 0.8 \times \text{Max_Retries}$, elevate severity.

4. **Step 4:** Check Clustering. If $\text{count}(e_{type})$ in $\text{window}(T_{now} - 60s) > \theta$, elevate to **CRITICAL**.

B. Drift Detection Logic

We use two windows: **Baseline (W_b)** and **Recent (W_r)**.

$$Drift(\Delta) = \frac{\mu(W_r) - \mu(W_b)}{\sigma(W_b)}$$

Where:

- μ = Mean dead-event ratio.
- σ = Standard deviation of the baseline.
- If $\Delta > 2.0$, a **Reliability Drift Alert** is triggered.

8. Data Flow Visualization

```
sequenceDiagram
    participant S as Microservice
    participant I as Ingestor
    participant DB as Event Store
    participant SE as Severity Engine
    participant DA as Drift Analyzer
    participant UI as Dashboard

    S->>I: Send Failure Event (id: 101, retry: 3)
    I->>DB: Store Immutable Record
    DB->>SE: Trigger Severity Assessment
    SE-->>SE: Apply Deterministic Rules
    SE->>DB: Update Severity (MEDIUM)
```

```
    loop Every 5 Minutes
        DA->>DB: Fetch Window (Last 1hr)
        DA-->>DA: Compare vs Baseline
        DA->>UI: Update Drift Metric (+15%)
    end
```

9. Visual Insights (System Metrics)

Performance Impact: Detection Latency

As event volume increases, AERIS maintains sub-100ms processing time due to the lightweight rule-based engine.

```
Latency (ms)
|
|          /--- Probabilistic ML (Opaque)
|         /
|  ______/----- AERIS Deterministic (Linear)
| /
|/___________________ Event Volume (req/sec)
```

Reliability Drift Visualization

The following chart represents a "System Health" view where the gap between Baseline and Current indicates the risk level.

```
Failure Rate (%)
   ^
10 |          [Alert!]
   |            / \
```

```
5 |  Baseline /   \  Current Behavior
  |  ~~~~~~~~/     \~~~~~~~~
0 +----------------------------> Time
```

10. Example Scenario: Payment Gateway Failure

Scenario: A microservice handling "Stripe Payments" starts failing due to an expired API key.

1. **Event 1:** Retry 1 fails. AERIS logs it. Severity: **LOW** (Temporary hiccup).
2. **Event 10:** Retry 5 (Max) fails. Event moves to Dead Letter. AERIS updates Severity to **HIGH**.
3. **Clustering:** Within 2 seconds, 50 different Payment events hit Max Retries.
4. **AERIS Logic:** "Multiple distinct entities reaching Max Retries in < 2s."
5. **Output: CRITICAL**. Reason: *Cluster detected + Dead Letter Exhaustion*.
6. **Drift:** The Dashboard shows the "Dead Event Ratio" jumped from 0.01% to 8% in 5 minutes. Confidence: **98%** (High sample size).

11. Advantages & Limitations

Advantages

- **Auditability:** Every failure decision can be defended in a post-mortem.
- **Low Overhead:** No complex model training required.
- **SRE Trust:** Provides the "Why" alongside the "What."

Limitations

- **Scale:** SQLite is suitable for research/small-scale; a distributed system would require migrating to a time-series DB like TimescaleDB.
- **Static Rules:** Rules require domain knowledge to set up initially.

12. Future Improvements

- **Dynamic Rule Evolution:** Use historical drift data to suggest new severity thresholds.
- **Cloud Ingestors:** Native adapters for AWS SQS and Azure Service Bus.
- **Multi-tenant Intelligence:** Separating reliability logic by service domain.

13. Conclusion

AERIS provides a robust framework for understanding the "ghosts in the machine" of event-driven systems. By treating failure as structured data and applying deterministic logic, we bridge the gap between simple monitoring and true reliability intelligence. This framework ensures that when a system fails, the recovery is guided by